Energy and Service-priority aware Trajectory Design for UAV-BSs using Double Q-Learning

Sayed Amir Hoseini[1,6], Ayub Bokani[2], Jahan Hassan[3], Shavbo Salehi[4] Salil S. Kanhere[5]
[1,2,3] School of Engineering and Technology, Central Queensland University, Sydney, Australia
[4] Electrical and Computer Engineering Department, Urmia University, Urmia, Iran
[5] School of Computer Science and Engineering, University of New South Wales, Sydney, Australia
[6] School of Engineering and Information Technology, University of New South Wales, Canberra, Australia
Email: {s.hoseini, a.bokani, j.hassan}@cqu.edu.au, shavbo.salehi@urmia.ac.ir, salil.kanhere@unsw.edu.au

2021 IEEE 18th Annual Consumer Communications & Networking Conference (CCNC) | 978-1-7281-9794-4/21/$31.00 ©2021 IEEE | DOI: 10.1109/CCNC49032.2021.9369472

***Abstract*—Next generation mobile networks have proposed the integration of Unmanned Aerial Vehicles (UAVs) as aerial base stations (UAV-BS) to serve ground nodes. Despite the advantages of UAV-BSs, their dependence on the on-board, limited-capacity battery hinders their service continuity. Shorter trajectories can save flying energy, however UAV-BSs must also serve nodes based on their service priority since nodes' service requirements are not always the same. In this paper, we present an energy-efficient trajectory optimization for a UAV assisted IoT system in which the UAV-BS considers the IoT nodes' service priorities in making its movement decisions. We solve the trajectory optimization problem using Double Q-Learning algorithm. Simulation results reveal that the Q-Learning based optimized trajectory outperforms a benchmark algorithm, namely Greedily served algorithm, in terms of reducing the average energy consumption of the UAV-BS as well as the service delay for high priority nodes.**

I. Introduction

Next generation wireless communication systems, e.g., 5G and beyond, have proposed the integration of Unmanned Aerial Vehicles (UAVs) as aerial base stations (UAV-BS) to provide terrestrial communication services [1]. Due to their dynamic deployability and mobility, UAVs can act as flying base stations to collect sensor data by moving closer to the Internet of Things (IoT) sensor nodes in remote farming or disaster areas without cellular or Wi-Fi coverage. An implementation of such a UAV-based sensor data collection system in the rural area of China for agricultural monitoring has been reported in [2]. However, unlike the terrestrial base stations that have continuous power supply, the UAV-BSs, or UAVs in general, have limited power supply from their on-board battery. For example, off-the-shelf drones such as DJI Spreading Wings S900 can stay afloat for around 18 minutes when fully charged [1]. Once the power is drained, the service provided by the UAV needs to pause for battery replacement or recharging. This leads to service-time limitations and discontinuity of UAV-delivered services.

The aerial recharging of UAVs can be considered as one of the solutions to address the power limitation issue [3], [4]. However, it certainly motivates the design of methods and mechanisms for a more energy-efficient functionality. Such efficiency allows them to do more within their battery budget, irrespective of the available recharging mechanisms. Since the major energy consumption of a UAV comes from its mechanical actions, e.g., flying, shortening the flight duration in its mission is an effective way to reduce energy consumption, which inevitably leads to a trajectory optimization problem. While conserving energy is highly important, in a UAV-based IoT sensor data collection scenario, the UAV must also serve ground sensors as per their priority or urgency levels, e.g., based on their delay tolerance levels [5]. Therefore, while optimizing the trajectory for energy-savings, the nodes' priorities must be considered by the UAVs. Related works studied these two criteria independently; for instance, [6], [7] proposed an energy-efficient trajectory optimization, and [5], [8] proposed nodes' priority-based trajectory optimization. In particular, authors in [8] presented a UAV-assisted data acquisition framework for sensor networks. The sensor nodes have different transmission priorities based on their locations relative to the UAV, which controls transmission orders, resulting in throughput maximization. Wang et al. [5] presented a Deep Q-Learning based trajectory optimization for time-sensitive UAV-assisted IoT network, where the sensor nodes' priorities are set based on their latency tolerance sensitivity. Authors aimed to minimize the overall system latency for UAV data collection.

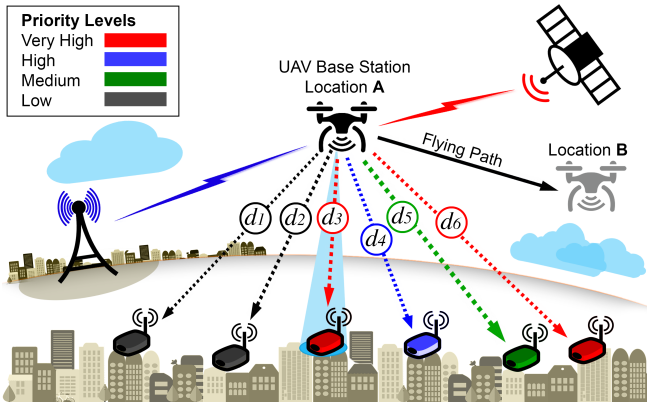


Fig. 1: **UAV assisted IoT System** in which the UAV serves nodes. E.g., IoT devices or sensors, with different priority levels and distances. Nodes should be in communication range, therefore, the UAV will fly above them at a certain distance to collect data.

In our previous work [9], the UAV's energy efficiency was optimized using the Travelling Salesman Problem (TSP)

Authorized licensed use limited to: Hanoi University of Science & Technology. Downloaded on March 12,2026 at 08:50:23 UTC from IEEE Xplore. Restrictions apply.

method for trajectory design, but the applicability of TSP is limited when the service priory needs to be considered as well. In this paper, therefore, we design a UAV trajectory that minimizes flying costs while serving nodes according to their priorities in a UAV assisted IoT system. An intelligent model is required for the UAV to make the best node-visiting decisions at different states. The problem can be formulated as that of selecting an action from a finite set of choices based on the repetitive observations of the environment. We optimize the UAV's path using Double Q-Learning [10], a model-free reinforcement learning algorithm. Q-Learning not only learns in which order the nodes should be served by the UAV after some experiences, but also can dynamically update the decision policy if the environment or nodes' behaviour changes. Our contributions in this paper are summarized below:

- We demonstrate that the UAV trajectory can be optimized based on energy-efficiency and service-priority using Double Q-Learning in a UAV-assisted sensor data collection scenario, and evaluate the performance of the proposed system using simulations.
- Our simulation results confirm that the optimized trajectory not only achieves significant energy savings for the UAV compared to the bench-marking algorithm, but also serves nodes as per their priorities, thereby enhancing the practicality of such systems.

II. System Model and Q-Learning Formulation

In our considered scenarios, we assume randomly positioned static ground nodes with different priority levels that require data collection services from the UAV as shown in Figure 1. Priority values change as per nodes' residual energy level. Nodes' location and initial service priority are pre-loaded in UAV software. Each time the UAV collects data from a node, it learns the node's updated residual energy level and once all nodes are served, the new energy level values are used by the UAV to determine the priorities in the next data collection round.

We model the service area on the ground as a grid and consider the UAV as the Q-Learning agent. The Q-Learning model consists of following four basic components:

- **Agent** (UAV) is the flying base station which should serve nodes one by one.
- **Action** (a) is the UAV's next flying destination which is determined by the location of next node to serve.
- **State** (s) is defined based on the observed information of UAV's current location and nodes information. Therefore, our system state is defined as $S = \{L_{uav}, L_{nd}, \Omega_{nd}\}$ where L_{uav} is UAV's location, $L_{nd} = [L_{nd_1}, L_{nd_2}, ..., L_{nd_n}]$ is a vector that represents the location of node 1 to n and $\Omega_{nd} = [\Omega_{nd_1}, \Omega_{nd_2}, ..., \Omega_{nd_n}]$ is a vector that denotes nodes' service priority (sp) and status (i.e., served or not). We consider a number from 1 to 4 to represent low, medium, high and very high sp levels respectively. Once a node is served, its sp level is set to zero, indicating that it has been served.
- **Revenue** (R) is a function that returns a real number for each $state - action$ pair after the Q-Learning agent has moved to the next state s' by taking action a. Our revenue function is a linear combination of $rewards$ and $penalties$.

In this model, we consider variable time steps due to having different distances between the UAV and each ground node. At each time step, the UAV selects the next waiting node and flies towards it. The service time is assumed to be negligible and the UAV is assumed to fly at a fixed speed and quickly communicates with nodes. To save ground nodes' energy, the UAV communicates data in the closest distance with these devices which means the UAV collects data when it is on top of them. While traditional Q-Learning uses one Q-Table, the double Q-Learning employs two Q-Tables to avoid possible local optima and achieves the global optimum. Hence, we denote these Q-Tables as Q_A-Table and Q_B-Table. After serving each node, the Q-values of the Q-table that was used in serving the node are updated using the relevant double Q-Learning equation from the below equations:

$$Q_A^{new}(s,a) = (1-\alpha)Q_A(s,a) + \alpha(R(s,a) + \gamma Q_B(s',a^*)), \tag{1}$$

$$Q_B^{new}(s,a) = (1-\alpha)Q_B(s,a) + \alpha(R(s,a) + \gamma Q_A(s',b^*)), \tag{2}$$

where α and γ are the learning rate and discount factor respectively. R is the revenue function, s$'$ is the next state after taking action a at state s and a^* and b^* are the maximum Q-value of all state-action pairs on state s$'$ as:

$$a^* = \arg\max_a Q_A(s',a), \tag{3}$$

$$b^* = \arg\max_a Q_B(s',a). \tag{4}$$

For the Q-Learning revenue function $R(s,a)$, we consider a *reward* when the UAV collects data from a high priority node and we apply different $penalties$ for the service delivery delays and flying energy consumption. Hence, The revenue function is calculated as:

$$R = w_1\Omega_{nd_a} - w_2 \sum_{i=1, i\neq a}^{n} \Omega_{nd_i} t_s - w_3 \int_0^{t_s} P(V)dt, \tag{5}$$

where w_1, w_2 and w_3 are the tuning parameters, nd_a is the served node, and t_s is elapsed time from serving the last node. $P(V)$ is UAV's power consumption for flying with speed V which is calculated as [6]:

$$P(V) = P_0\left(1 + \frac{3V^2}{U^2}\right) + P_i\left(\sqrt{1 + \frac{V^4}{4\nu_0^4}} - \frac{V^2}{2\nu_0^2}\right)^{1/2} + \frac{1}{2}d_0\rho s A V^3 \tag{6}$$

where relevant parameters are listed in Table I.

We use *epsilon-greedy* scheme [11] in our algorithm where actions are taken randomly at the beginning of learning process and the agent is fully in *exploration* mode. Besides, by decreasing the epsilon value, the chance of *exploitation* increases and the action with the highest Q-Value may be taken

Authorized licensed use limited to: Hanoi University of Science & Technology. Downloaded on March 12,2026 at 08:50:23 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 : Algorithm for Double Q-Learning Episode

```
Load Q_A-Table and Q_B-Table from previous episode
Initialize UAV location
exploration = epsilon
Observe nodes' location
Set Q-Selector = A
while there is a node to serve
do {
  Observe nodes' service priority
  Current state = (UAV location, nodes' location, nodes' priority)
  Generate a random number p ∈[0,1]
  if p is less than Exploration
      Select next node randomly
  else
      if Q-Selector = A
          Choose next node based on Q_A-Values
      else if Q-Selector = B
          Choose next node based on Q_B-Values
  Move UAV to next node location
  Current node = Next node
  Collect current node data
  Calculate Revenue(consumed energy by UAV,
   current node service priority, service delay)
  if Q-Selector = A
      Update Q_A-Table using (1)
  else if Q-Selector = B
      Update Q_B-Table using (2)
  Set current node service priority to zero
  Toggle Q-Selector between A and B
  }
Decrease epsilon for the next episode
```

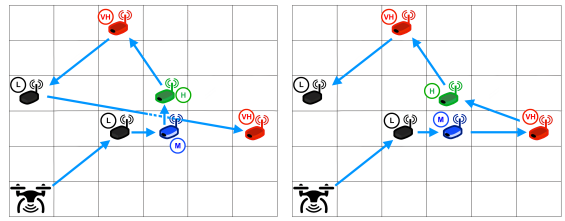
at each step. This is adjusted to rely on Double Q-Learning policy gradually over the time.

At each step of time, UAV observes the state s, then takes an action a and receives a revenue $R(s, a)$ after moving to the state s'. The goal of the training phase is to find the sequential order of served nodes that maximizes the total future revenues. An episode of Double Q-Learning is presented in Algorithm 1. The $Q_A(s, a)$ and $Q_B(s, a)$ values are saved and updated in Q_A-Table and Q_B-Table.

To compare Double Q-Learning performance, we consider a Greedy algorithm as a baseline. The Greedy (Nearest Neighbor) is presented in Algorithm 2 in which the UAV selects the nearest node to serve at each step irrespective of node priority, whereas the Double Q-Learning tries to achieve a balance between distance and node priority.

Algorithm 2 : Greedy Algorithm

```
Initialize UAV location
Observe nodes' location
while there is a node to serve
do {
  remaining nodes = nodes with non-zero service priority
  Calculate the distance between UAV and remaining nodes
  Select the node with minimum distance as next node
  Move UAV to next node location
  current node = next node
  Collect current node data
  Set current node service priority to zero
  Decrease Exploration
  }
```



(a) Greedy Algorithm (b) Double Q-Learning

Fig. 2: UAV base station trajectory for scenario #1.

TABLE I: Simulation Parameters

Cell side	50 m
Learning rate (α)	0.5
Discount factor (γ)	0.95
Revenue tuning parameter (w_1)	30
Revenue tuning parameter (w_2)	7.5 s^{-1}
Revenue tuning parameter (w_3)	0.1 Joule^{-1}
Air density (ρ)	1.225 $\mathrm{kg/m^3}$
Rotor disc area (A)	0.181 $\mathrm{m^2}$
Tip speed of the rotor blade (U)	96 $\mathrm{m/s}$
Fuselage drag ratio (d_0)	0.9
Rotor solidity (s)	0.1
UAV speed (V)	5 $\mathrm{m/s}$
Blade profile power (P_0)	29.4 W
induced power (P_i)	206.5 W
Mean rotor induced velocity (ν_0)	7.5 $\mathrm{m/s}$

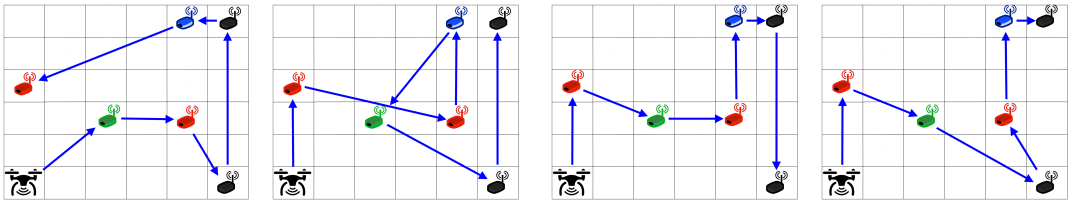
III. Performance Evaluation

To evaluate the proposed method, we simulated two scenarios as illustrated in Figures 2 and 3 respectively, where six nodes are randomly distributed on a 6 by 6 grid service area. Each node with a random service priority represented by different colours as was illustrated in Figure 1, has data waiting for collection. As mentioned above, ϵ-greedy scheme is used to force Double Q-Learning to take random actions at the start which is helpful in training to avoid local convergence. Simulation parameters are shown in Table I.

The first scenario is simulated using default tuning parameters of revenue function for Double Q-Learning and Greedy algorithm. After around 5000 episodes, our Double Q-Learning model outperforms the Greedy algorithm for both revenue and energy consumption. Figure 2 shows the resulting trajectory for both of the algorithms, using the same priority color coding for nodes as was shown in Figure 1. The UAV starts flying from the bottom left corner and collects nodes' data one by one. Note that a near-optimal trajectory policy can be obtained offline by Double Q-Learning and utilized from the beginning of flight missions and gets updated by experiencing real-world operations continuously.

The revenue function in (5) has an essential role in Double Q-Learning behavior. The trajectory optimisation of UAV in our model depends to service priority of nodes, service delay and energy consumption of UAV. Therefore, there is a trade-off which can be adjusted by the revenue function. To explore this, we simulated the second scenario for three different tuning parameter set of the revenue function. In each set, we choose a very large value for one of w_1, w_2 and w_3. We also simulate the Greedy algorithm for the same scenario for

Authorized licensed use limited to: Hanoi University of Science & Technology. Downloaded on March 12,2026 at 08:50:23 UTC from IEEE Xplore. Restrictions apply.



(a) Greedy algorithm

(b) DQL1: High revenue for service to high priority node. $w_1 = 30000$, $w_2 = 7.5$, $w_3 = 0.1$.

(c) DQL2: High penalty for priority-weighted delay.$w_1 = 30$, $w_2 = 750$, $w_3 = 0.1$

(d) DQL3: High penalty for energy consumption. $w_1 = 30$, $w_2 = 7.5$, $w_3 = 100$.

Fig. 3: Greedy and Double Q-Learning Trajectories for scenario #2. Double Q-Learning results are generated for different revenue adjustment parameters.

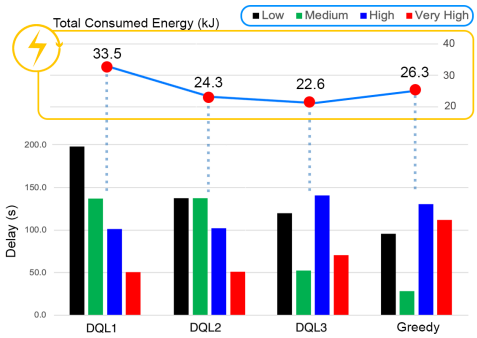


Fig. 4: Average Serving Delay and Energy Consumption of various revenue parameter sets for scenario #2.

comparison, with results presented in Figure 3 and Figure 4. The trajectory is plotted and the average delay is calculated after Double Q-Learning converges to a stable outcome for the fixed scenario. While the Greedy algorithm selects node service order based on the nearest neighbor approach, Double Q-Learning results in a different trajectory depending the parameter adjustments. When w_1 is set to very high value as in Figure 3b (DQL1), UAV collects data of high priority nodes first. This is changed to a balanced behavior when a very high w_2 is set as in Figure 3c (DQL2). As shown in Figure 4, the balanced Double Q-Learning presents an overall more efficient energy consumption. While serving high priority nodes results in 33.5 kJ of energy consumption, this is reduced to 24.3 kJ for the balanced mode. Further, the delay for serving low and moderate priority nodes are also reduced for the balanced Double Q-Learning in comparison to serving high priority nodes settings. Finally, we set w_3 to a very high value to minimize energy consumption. The result is presented in Figure 3d (DQL3). As can be seen in Figure 4, UAV is forced to travel the near shortest path and consumes only 22.6 kJ of energy which comes at a cost of high experienced delay for the higher priority nodes.

IV. Conclusion

We optimized the UAV's trajectory using Double Q-Learning, to reduce energy consumption while serving requesting nodes as per their required service priority. Simulation results revealed that the Q-Learning based trajectory outperformed the benchmark node-serving algorithm in reducing the average energy consumption as well as service delivery delay of the UAV for high priority nodes. We note that Double Q-Learning has performance limitations when applied to an extended UAV observation space, e.g., when the number of nodes increase. For a more scalable model, in our future work we aim to use Deep Reinforcement Learning (DRL), where we can employ the Deep Neural Network to estimate Q-Values in a large Q-Table. However, it is worth avoiding the complexity and instability of DRL [12] for small scale applications for which Double Q-Learning's efficiency is proven in this work.

Acknowledgment

This work is supported by the Central Queensland University Research Grant RSH5137.

References

[1] A. Fotouhi, H. Qiang *et al.*, "Survey on UAV cellular communications: Practical aspects, standardization advancements, regulation, and security challenges," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 4, 2019.

[2] G. Sylvester, *E-agriculture in action: Drones for agriculture*. Bangkok, Thailand: Food and Agriculture Organization of the United Nations and International Telecommunication Union, 2018.

[3] J. Hassan, A. Bokani *et al.*, "Recharging of flying base stations using airborne RF energy sources," in *IEEE WCNC Workshop (WCNCW)*, 2019.

[4] S. A. Hoseini, J. Hassan *et al.*, "Trajectory optimization of flying energy sources using Q-learning to recharge hotspot UAVs," in *IEEE INFOCOM Workshops*, 2020.

[5] N. Wang, Y. Xin *et al.*, "Priority-oriented trajectory planning for UAV-aided time-sensitive IoT networks," in *IEEE ICC Workshops*, 2020.

[6] Y. Zeng, J. Xu *et al.*, "Energy minimization for wireless communication with rotary-wing UAV," *IEEE Trans. Wireless Commun.*, vol. 18, no. 4, 2019.

[7] M. Bliss and N. Michelusi, "Power-constrained trajectory optimization for wireless UAV relays with random requests," in *2020 IEEE ICC*, 2020.

[8] S. Say, H. Inata *et al.*, "Priority-based data gathering framework in UAV-assisted wireless sensor networks," *IEEE Sensors J.*, vol. 16, no. 14, 2016.

[9] S. Salehi, A. Bokani *et al.*, "AETD: An application aware, energy efficient trajectory design for flying base stations," in *IEEE MICC*, December 2019.

[10] H. V. Hasselt, "Double Q-learning," in *Advances in neural information processing systems*, 2010, pp. 2613–2621.

[11] R. S. Sutton and A. G. Barto, *Reinforcement learning: An introduction*. MIT press, 2018.

[12] O. Anschel, N. Baram *et al.*, "Averaged-DQN: Variance reduction and stabilization for deep reinforcement learning," in *International Conference on Machine Learning*, 2017, pp. 176–185.

Authorized licensed use limited to: Hanoi University of Science & Technology. Downloaded on March 12,2026 at 08:50:23 UTC from IEEE Xplore. Restrictions apply.